

# Measuring the Defenders: A Layer-Aware, Framework-Mapped Benchmark for Model Context Protocol Security Proxies

Anonymous Author(s)

## Abstract

The Model Context Protocol (MCP) has become a common way to connect large-language-model (LLM) agents to external tools and data, and a substantial 2025–2026 literature now documents its attack surface. Existing security benchmarks for MCP measure how well an *LLM agent resists* attacks. None, to our knowledge, measure how well a *defensive proxy or gateway covers* that attack surface, and none map their results to the governance frameworks organizations actually use—the NIST AI Risk Management Framework (AI RMF) and the OWASP Top 10 for LLM and Agentic Applications. We present two artifacts to fill this gap: (1) a *threat–control crosswalk* of 24 MCP attack vectors mapped to architectural layer, STRIDE class, NIST AI RMF function, OWASP category, and the NSA MCP Security guidance; and (2) an open, vendor-neutral *defense-side benchmark* that runs a candidate MCP security proxy against a corpus of attack fixtures with matched benign controls, and scores its coverage using the crosswalk as a rubric. Each tool is driven through its *real* code, not a mock. We evaluate three open-source proxies (OurProxy, mcp-firewall, pipelock) plus a null baseline over 35 test cases. The strongest proxy reaches **63% weighted coverage (15.0 of 24 vectors; 11 enforced)**, and across all three tools **19 of 24 vectors are covered by at least one tool while 5 are covered by none**: the entire registry/supply-chain surface, OS-isolation-dependent vectors, and several semantic vectors. All results carry zero false positives against matched benign controls. We argue this is direct, quantified evidence that a runtime proxy alone is insufficient and that defense-in-depth spanning *distinct mechanism classes* (proxy, cryptographic attestation, OS isolation) is required. We further show the benchmark can *guide* a defender’s development—over the study, OurProxy’s coverage rose from 9% to 63% as gaps it surfaced were fixed and re-verified—including two attacks first published in June 2026, one of which (ShareLock) no measured tool covered until a cross-tool correlation check was added. A matched benign-control design, a “measure only what the tool inspects at runtime” integrity rule, and an automated benchmark-regression gate keep the results honest and reproducible.

## CCS Concepts

• **Security and privacy** → **Software security engineering**; • **Computing methodologies** → **Artificial intelligence**.

## Keywords

Model Context Protocol; MCP; AI security; LLM agents; agentic AI; security benchmark; tool poisoning; NIST AI RMF; NSA MCP Security; OWASP LLM Top 10; OWASP Agentic Top 10; defense-in-depth

## 1 Introduction

MCP standardizes how an LLM agent discovers and calls external “tools” exposed by “servers.” Its rapid adoption across consumer and

enterprise agents has been accompanied, in 2025–2026, by a dense body of security research: systematization-of-knowledge papers, formal threat taxonomies, attack benchmarks, and cryptographic attestation proposals (Section 2). This work establishes *what can go wrong*.

A practical question remains under-addressed: **given a defensive tool that sits in front of MCP servers—a proxy, gateway, or scanner—how much of the known attack surface does it actually cover?** Answering it well requires three things that, individually, exist in the literature but have not been combined: (1) a *shared taxonomy* of MCP attack vectors; (2) a mapping from those vectors to the *governance frameworks* (NIST AI RMF, OWASP) that security and procurement teams reason in; and (3) an *executable, reproducible* way to test a defender and report coverage.

We contribute all three, plus an empirical study:

- **A threat–control crosswalk** (Section 3): 24 MCP attack vectors, each mapped to one or more of six architectural layers, a STRIDE class, NIST AI RMF functions, NSA MCP Security recommendation areas, and OWASP LLM-2025 and Agentic-2026 categories. Published as open, versioned, machine-readable data.
- **A defense-side benchmark** (Section 4): for each vector, one or more attack *fixtures* paired with a near-identical *benign control*; a small *adapter* per tool that drives the tool’s real detection code; and a scorer that reports coverage per layer and per framework.
- **An empirical evaluation** (Sections 5–6) of three open-source proxies and a null baseline, yielding the complementarity finding above.
- **A candid methodology discussion** (Sections 7–8): the corpus-encoding sensitivity we observed, the matched-control and runtime-integrity safeguards, and the limits of the study.

Our aim is not to rank products but to map coverage, expose gaps, and provide reusable infrastructure.

## 2 Related Work

All 2603/2604-series works below are recent, non-peer-reviewed preprints whose taxonomies are their authors’ own constructs; we cite them as *proposed by*, not as established consensus. Coverage statistics quoted from any single paper reflect that paper’s surveyed set.

*Threat taxonomies.* Recent work systematizes MCP threats through several lenses: an SoK separating adversarial security threats (indirect prompt injection, tool poisoning) from epistemic safety hazards across the Resources/Prompts/Tools primitives [3]; a formal framework of 7 threat categories and 23 attack vectors grounded in a large tool corpus [1]; a STRIDE/DREAD model over five MCP components that identifies tool poisoning as the most impactful client-side

vulnerability [4]; and a defense-placement taxonomy classifying defenses by the layer that enforces them, which finds protection “uneven and predominantly tool-centric” [11]. Our crosswalk reuses these vector families but adds the framework mapping none of them provides.

*Attack-side benchmarks.* MSB [16] (12 attacks, ~2,000 instances, real MCP execution) measures the resistance of the *LLM agent* across the tool-use pipeline; MCPTox [15] scopes to tool poisoning; AgentDefense-Bench [12] aggregates datasets into MCP-JSON-RPC test cases and scores a defender’s *detection accuracy*; MCP-Bench [14] measures agent *capability*, not security. All measure the agent (or scheme accuracy), not a *defensive proxy’s coverage*. Ours measures the defender’s coverage, on the full mapped surface, crosswalked to NIST/OWASP—which we could not find in any surveyed benchmark.

*Attestation and provenance.* ETDI [2] proposes signed, versioned tool definitions; a “Trustworthy MCP Registry” blueprint [7] composes existing standards (well-known URIs, Sigstore, JSON Canonicalization + JWS) for provenance and runtime integrity, and notes the official MCP registry remains an unverified pointer architecture. These are complementary to our benchmark; a future attestation-conformance suite (Section 9) would test them.

*Scanners and gateways.* Practitioner tools include Invariant Labs’ MCP-Scan (since acquired by Snyk and now a cloud-gated scanner [13]), Lasso’s and EnkryptAI’s MCP gateways, Docker’s isolation-based MCP gateway, and the proxies we evaluate here. Our benchmark scores locally-deterministic defenders and treats cloud-model defenders as observable-but-not-reproducibly-scoreable (Section 4.4).

### 3 The Threat–Control Crosswalk

The crosswalk is the benchmark’s rubric and a standalone artifact. It enumerates **24 MCP attack vectors** consolidated from the taxonomies in Section 2, and maps each to: *architectural layer* (host-orchestration, client, transport, server, tool, registry-supply-chain), following the defense-placement view of [11]; *STRIDE* class; *NIST AI RMF* function(s); *NSA MCP Security* recommendation area [8]; *OWASP Top 10 for LLM Applications (2025)* [10]; and *OWASP Top 10 for Agentic Applications (2026)* [9]. Table 1 lists the vectors and primary layer; the full mapping is published as machine-readable data (rubric/crosswalk.json, CC-BY-4.0).

The mappings are indicative and reviewable, not a certification—we invite community correction, itself a goal of publishing the rubric openly. The OWASP Agentic categories used are the official 2026 set (ASI01–ASI10).

## 4 Benchmark Methodology

### 4.1 Design

For each vector the corpus contains one or more *test cases*. A test case is a static JSON object with: a *malicious fixture* (the attack, as data—a tool definition, tool call, tool result, registry entry, or scenario), a near-identical *benign control*, the *expected* defender behavior (detect/enforce), and a *provenance* note. Fixtures are static data, never live exploits; attacker hosts use reserved names

(the cloud-metadata address 169.254.169.254 appears only as an SSRF *target* in data, never contacted).

### 4.2 Adapters and the runtime-integrity rule

Each evaluated tool provides a thin *adapter* exposing `assess(input) → {detect, enforce}`. The runner calls it once on the malicious fixture and once on the benign control. Adapters drive the tool’s *real* code (importing the proxy’s detection functions, or invoking its CLI/SDK with a committed configuration). A central rule keeps scores honest:

**Report only what the tool actually inspects at runtime—never what its rules *could* match if pointed at data the tool never sees.**

For example, a proxy that inspects tool *definitions* but not tool *results* returns “not detected” for result-borne attacks, even if its text rules would match the result string in isolation. This measures deployed behavior, not theoretical capability.

### 4.3 Scoring

Per test case: if the malicious fixture is not flagged, the vector scores **none**; if it is flagged but the benign control is *also* flagged, the vector scores **none** and is marked a **false positive**; if the malicious fixture is flagged and the benign control is clean, the vector scores **detect** (warn) or **enforce** (block). A vector’s level is the best across its test cases. Weighted coverage assigns `enforce = 1.0`, `detect = 0.5`, `none = 0`, over 24 vectors. The matched benign control is essential: without it, a tool that flags everything would score perfectly while being useless.

### 4.4 Reproducibility rule

A defender is *scored* only if its detection logic runs *locally and deterministically*. Tools that delegate detection to a proprietary cloud model can be *observed* but not reproducibly *scored*, since their verdicts depend on a black box that can change between runs. This excludes, e.g., the cloud-gated Snyk agent-scan; it includes the three proxies below.

## 5 Experimental Setup

We evaluate four adapters over a 35-case corpus (24 base cases + 6 realistic “v2” cases for fairness + 3 “v3” evasion-robustness cases + 2 added with new vectors; see Section 7). **null-baseline** (control) returns “not detected” for all inputs. **OurProxy** (runtime proxy, Apache-2.0) is driven by importing its real detection code—definition/text scanning with evasion normalization, cross-tool correlation, cross-server taint, argument and command-injection validation, config-drift and server-identity checks, and inline secret redaction—under its default balanced enforcement profile. **mcp-firewall** (runtime proxy, Python, AGPL-3.0) is driven via its real SDK (`Gateway.check/scan_response`) with a committed config. **pipelock** (egress firewall, Go, Apache-2.0 core) is driven via its real scanners (`explain -json, mcp scan`), offline and deterministic. For AGPL/other licensed tools we *run* the tool to benchmark it and report scores; we do not redistribute their code. Each tool is pinned to a released version and configured with its own recommended/starter policy, committed for reproducibility. Table 2 summarizes the four adapters and how each is driven.

**Table 1: The 24 MCP attack vectors and their primary architectural layer.**

| #  | Vector                                    | Layer                 | #  | Vector                                 | Layer                 |
|----|-------------------------------------------|-----------------------|----|----------------------------------------|-----------------------|
| 1  | Tool poisoning                            | tool                  | 13 | Sandbox escape                         | server                |
| 2  | Tool shadowing / name collision           | client                | 14 | Schema / validation bypass             | server                |
| 3  | Rug pull (definition mutation)            | tool                  | 15 | Man-in-the-middle (transport)          | transport             |
| 4  | Out-of-scope parameter injection          | tool                  | 16 | DNS rebinding (local servers)          | transport             |
| 5  | Prompt injection via tool results         | tool                  | 17 | Server impersonation                   | registry-supply-chain |
| 6  | Indirect / retrieval injection            | tool                  | 18 | Excessive permission / escalation      | host-orchestration    |
| 7  | Cross-tool exfiltration (confused deputy) | client                | 19 | Credential / token theft               | host-orchestration    |
| 8  | Tool-transfer / cross-server chaining     | host-orchestration    | 20 | Consent fatigue / over-broad grants    | client                |
| 9  | False-error escalation                    | tool                  | 21 | Command injection                      | server                |
| 10 | Package / name squatting                  | registry-supply-chain | 22 | System-prompt / context leakage        | client                |
| 11 | Supply-chain poisoning (provenance gap)   | registry-supply-chain | 23 | Mid-session tool injection (MSTI)      | client                |
| 12 | Configuration drift                       | server                | 24 | Multi-tool split poisoning (ShareLock) | tool                  |

**Table 2: Evaluated adapters. Each defender is driven through its *real* code path under a committed configuration.**

| Tool          | Class / license             | Driven via                            |
|---------------|-----------------------------|---------------------------------------|
| null-baseline | control                     | returns “not detected”                |
| OurProxy      | runtime proxy, Apache-2.0   | real detection code, balanced profile |
| mcp-firewall  | runtime proxy, AGPL-3.0     | real (Gateway.check) SDK              |
| pipelock      | egress firewall, Apache-2.0 | real scanners (explain, mcp scan)     |

**Table 3: Verified coverage (35 cases; weighted over 24 vectors).**

| Tool          | Class           | Cov. | enf. | det. | none | FP   |
|---------------|-----------------|------|------|------|------|------|
| OurProxy      | runtime proxy   | 63%  | 11   | 8    | 5    | 0/35 |
| mcp-firewall  | runtime proxy   | 13%  | 3    | 0    | 21   | 0/35 |
| pipelock      | egress firewall | 10%  | 1    | 3    | 20   | 0/35 |
| null-baseline | control         | 0%   | 0    | 0    | 24   | 0/35 |

## 6 Results

A proxy’s ceiling is partial. OurProxy, the broadest tool, spans the definition, response, argument, transport, and egress/least-privilege layers. mcp-firewall and pipelock add *enforcement* (deny/redact/SSRF-block) on the call and egress layers where bastion only warns; pipelock additionally detects response-borne prompt injection via its MCP-response scanner.

Two newly-published attacks (June 2026). We added mid-session tool injection (MSTI [5]) and multi-tool split poisoning (ShareLock [6]) as vectors 23–24. MSTI—a tool that mutates its behavior after first approval, mid-session—is caught by OurProxy’s definition pinning, which re-hashes the tool on each use and enforces on a post-pin change; neither other tool covers it. ShareLock—a payload split across several tools so no single description looks malicious—was covered by *no* measured tool until we added *cross-tool correlation* to OurProxy (Section 7): because a proxy sees a server’s whole tool set, it scans combined descriptions and flags co-ordinated staging metadata across tools. It now *detects* ShareLock;

the other two, which scan one tool at a time, still miss it. This is a staging-signal heuristic, not a cryptographic defeat of threshold secret-sharing (Section 8).

*Defense-in-depth across mechanism classes is required.* Even with the strongest proxy at 63%, and across all three tools (Table 4), **19 of 24 vectors are covered by at least one tool; 5 are covered by none**—false-error escalation, package/name squatting, provenance-gap supply-chain poisoning, sandbox escape, and consent fatigue. These are not addressable by *any* content-scanning runtime proxy; they require different mechanisms—cryptographic attestation and a verified registry, OS sandboxing, and client-side consent controls. A proxy is necessary but not sufficient.

*Evasion robustness.* The corpus includes three obfuscated variants—zero-width/bidi-control characters, Cyrillic homoglyph substitution, and base64-wrapped payloads. After adding a normalization stage (Unicode NFKC, homoglyph folding, base64 decode-and-rescan), OurProxy catches all three (3/3); pipelock catches homoglyph and base64 but misses zero-width (2/3); mcp-firewall catches none.

*Zero false positives.* No tool flagged any benign control, across all 35 cases and every feature iteration. Enforcement level (block vs. warn) does not change *whether* a control is flagged, so promoting deterministic checks to blocking preserves the zero-false-positive property.

## 7 Discussion

*Benchmark-guided improvement (9%→63%).* The benchmark initially scored OurProxy at 9%, covering only the tool-definition layer. It then guided several rounds of proxy-native hardening, each re-measured at zero false positives: response scanning (9→18%), argument/schema validation (18→23%), transport hardening (23→30%), sensitive-argument plus least-privilege scanning (30→34%), and cross-tool correlation (34→38%). Two further release cycles took it to 63%: four new proxy-native checks—command-injection, configuration drift, server-identity pinning, and cross-server data-flow (taint) tracking—each closing a vector no measured tool covered (38→48%); and a “depth” cycle (48→63%) adding evasion normalization, inline DLP redaction, and a confidence-tiered enforcement profile whose default *blocks* the deterministic, near-zero-false-positive checks—so the proxy now **blocks 11 of the 19 vectors it covers**. The

**Table 4: Per-vector coverage over the 24-vector surface. E = enforce (blocks), D = detect (warns), – = none. O = OurProxy, F = mcp-firewall, P = pipelock. OurProxy covers 19/24 (11 E, 8 D); the five all–“–” rows (9, 10, 11, 13, 20) are covered by no measured tool and lie outside any content-scanning proxy’s reach.**

| #  | Vector                     | O | F | P | #  | Vector                   | O | F | P |
|----|----------------------------|---|---|---|----|--------------------------|---|---|---|
| 1  | Tool poisoning             | D | – | – | 13 | Sandbox escape           | – | – | – |
| 2  | Tool shadowing             | D | – | – | 14 | Schema/validation bypass | E | – | – |
| 3  | Rug pull                   | E | – | – | 15 | MITM (transport)         | D | – | – |
| 4  | Out-of-scope param inj.    | E | – | – | 16 | DNS rebinding            | E | – | – |
| 5  | Prompt inj. via results    | D | – | D | 17 | Server impersonation     | E | – | – |
| 6  | Indirect/retrieval inj.    | E | E | D | 18 | Excessive permission     | D | – | – |
| 7  | Cross-tool exfiltration    | E | E | E | 19 | Credential/token theft   | E | E | – |
| 8  | Tool-transfer/cross-server | E | – | – | 20 | Consent fatigue          | – | – | – |
| 9  | False-error escalation     | – | – | – | 21 | Command injection        | E | – | – |
| 10 | Package/name squatting     | – | – | – | 22 | System-prompt leakage    | D | – | D |
| 11 | Supply-chain poisoning     | – | – | – | 23 | Mid-session tool inj.    | E | – | – |
| 12 | Configuration drift        | D | – | – | 24 | Multi-tool split poison. | D | – | – |

benchmark did not just measure the tool; it identified concrete, layer-specific gaps, verified each fix, and then defined the proxy’s ceiling.

*A living benchmark tracks new attacks.* Vectors 23–24 were published (June 2026) *after* the v0.2 rubric was frozen; adding them exercised the benchmark’s intended lifecycle and immediately exposed that ShareLock was covered by none, driving the cross-tool-correlation fix. A benchmark that only measured a fixed attack set would have missed the gap entirely.

*Tool-class taxonomy (score vs. track).* *Content-scanning proxies* (the three here) inspect tool definitions, calls, or results at runtime; the rubric measures exactly what they inspect, so we *score* them. *Access-control and isolation gateways* enforce identity, policy, and sandboxing rather than scanning content; the rubric’s content-borne vectors would score them near zero not because they are weak but because they operate on a different axis. We *track* these without a score; a class-appropriate rubric is future work. Reporting a misleading number would be worse than reporting none.

*Corpus-encoding sensitivity (a fairness hazard).* Scores are sensitive to how an attack is *encoded*. An early corpus expressed exfiltration with sanitized placeholder text; against it mcp-firewall scored 5%. Adding realistic encodings (an AWS-key-shaped secret; an exfiltration POST to the cloud-metadata address) raised it to 13%—not by changing the tool, but by probing capabilities the first corpus never exercised. We therefore treat tool-neutral, realistic, multi-encoding fixtures as a fairness requirement, and caution that **absolute totals are corpus-dependent; the per-vector matrix (Table 4) is the more reliable read.**

*Adapter faithfulness.* The matched benign control also guards against adapter *over-reach*. In wiring pipelock (an egress firewall), an initial adapter synthesized an egress URL from an *inbound* local host, causing pipelock to “block” both the malicious and the benign case—surfaced immediately as a false positive and corrected by restricting URL synthesis to outbound endpoints. Without the benign twin, this adapter bug would have silently inflated pipelock’s score.

*Why coverage, not a single score.* Because absolute totals move with the corpus, the contribution is the *per-vector, per-layer map* (Table 4), not a leaderboard rank. Two tools with similar totals

can be complementary: mcp-firewall and pipelock each *enforce* on vectors where OurProxy only warns, so a deployment combining classes covers more than any single tool—the practical reading of “defense-in-depth” the numbers support.

## 8 Limitations and Threats to Validity

- **Taxonomy is a synthesis, not a standard.** The 24-vector set consolidates several proposed taxonomies; framework mappings are indicative and single-author.
- **Small, seeded corpus.** Most vectors have 1–2 fixtures; absolute coverage numbers are lower bounds and corpus-dependent.
- **The ShareLock check is a heuristic, not a proof.** It flags the *staging signal* the published attack uses, not a cryptographic defeat of threshold secret-sharing; reported as detect, not enforce.
- **Partial tool interfaces.** For pipelock we drive its offline URL/egress and MCP-response scanners, not its poisoned-description or full DLP endpoint; its number is a lower bound.
- **Configuration dependence.** Results depend on each tool’s configured policy, committed for reproducibility.
- **Scope and tool class.** We score locally-deterministic content-scanning proxies; cloud-model defenders and access-control/isolation gateways are out of scope for scoring.

## 9 Future Work

Expand the corpus to multiple tool-neutral fixtures per vector; deepen the evasion-robustness set; drive pipelock’s remaining interfaces and add further defenders; publish a public leaderboard; complete a second-reviewer pass on framework mappings; and build an *attestation-conformance* suite to test tool-integrity/provenance schemes (ETDI [2], the Trustworthy Registry composition [7]) as a coupled bench-plus-attestation artifact.

## 10 Conclusion

We introduced a layer-aware, framework-mapped crosswalk of 24 MCP attack vectors—mapped to NIST AI RMF, the OWASP LLM and Agentic Top 10s, STRIDE, and the NSA MCP Security guidance—and an open, vendor-neutral benchmark that measures how much

of that surface a defensive proxy actually covers, driving each tool through its real code. The strongest of three open-source proxies reaches 63% weighted coverage with 11 vectors enforced, and 5 of 24 vectors are undefended by any measured tool. The evidence argues for layered defense across distinct mechanism classes, and the benchmark demonstrably guided one proxy from 9% to 63% at zero false positives. It provides a reusable, honest instrument for the community to measure MCP defenders and track their progress as the threat surface evolves.

## Availability and Disclosure

The benchmark (rubric, corpus, adapters, runner, leaderboard; rubric/data CC-BY-4.0, harness Apache-2.0) and the reference proxy (Apache-2.0) are released open-source. For double-blind review the artifact is provided as an *anonymized mirror*: <https://anonymous.4open.science/r/OurBench/>. The de-anonymized repositories and archival DOIs will be provided at camera-ready. *Disclosure*: one evaluated tool was developed by the authors and is scored by the same harness alongside competitors and a null baseline.

## Use of Generative AI

Generative AI (a large-language-model assistant) was used in preparing this paper. Specifically, it assisted with formatting the manuscript into the ACM template, editing and condensing prose, and generating  $\LaTeX$  tables from the authors' own measured benchmark results. All research contributions—the benchmark design, the threat-control crosswalk, the experimental methodology, and every measurement and result—are the authors' own work. The authors reviewed and verified all AI-assisted text and tables; no experimental results or data were AI-generated, and all citations were verified against their primary sources.

## References

- [1] N. Acharya and G. K. Gupta. 2026. A Formal Security Framework for MCP-Based AI Agents: Threat Taxonomy, Verification Models, and Defense Mechanisms. arXiv:2604.05969 [cs.CR]
- [2] M. Bhatt, V. S. Narajala, and I. Habler. 2025. ETDI: Mitigating Tool Squatting and Rug Pull Attacks in Model Context Protocol (MCP) by using OAuth-Enhanced Tool Definitions and Policy-Based Access Control. arXiv:2506.01333 [cs.CR]
- [3] S. Gaire, S. Gyawali, S. Mishra, S. Niroula, D. Thakur, and U. Yadav. 2025. Systematization of Knowledge: Security and Safety in the Model Context Protocol Ecosystem. arXiv:2512.08290 [cs.CR]
- [4] C. Huang, X. Huang, N. P. Tran, and A. Milani Fard. 2026. Model Context Protocol Threat Modeling and Analyzing Vulnerabilities to Prompt Injection with Tool Poisoning. arXiv:2603.22489 [cs.CR]
- [5] L.-F. Lee, Y.-Y. Chang, C.-M. Yu, and K.-H. Yeh. 2026. WebMCP Tool Surface Poisoning: Runtime Manipulation Attacks on LLM Agents. arXiv:2606.06387 [cs.CR]
- [6] L. Liu, T. Han, Z. Liu, Z. Dong, and N. Ruan. 2026. ShareLock: A Stealthy Multi-Tool Threshold Poisoning Attack Against MCP. arXiv:2606.27027 [cs.CR]
- [7] L. Mas, J. Vilaplana, J. Rius, R. Spaimoc, and J. Mateo. 2026. The Trustworthy Model Context Protocol (MCP) Registry: An Architectural Blueprint for Cryptographic Provenance and Runtime Integrity. *Future Internet* 18, 5 (2026), 243. doi:10.3390/fi18050243
- [8] National Security Agency, AI Security Center. 2026. *MCP Security: Security Design Considerations for AI-Driven Automation*. Technical Report Cybersecurity Information Sheet U/OO/6030316-26. NSA. [https://www.nsa.gov/Portals/75/documents/Cybersecurity/CSI\\_MCP\\_SECURITY.pdf](https://www.nsa.gov/Portals/75/documents/Cybersecurity/CSI_MCP_SECURITY.pdf)
- [9] OWASP GenAI Security Project. 2025. OWASP Top 10 for Agentic Applications (2026). <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/>
- [10] OWASP GenAI Security Project. 2025. OWASP Top 10 for LLM Applications (2025). <https://genai.owasp.org/llm-top-10/>
- [11] M. Rostamzadeh, S. Narula, N. Birhan, M. Ghasemigol, and D. Takabi. 2026. MCP-DPT: A Defense-Placement Taxonomy and Coverage Analysis for Model Context Protocol Security. arXiv:2604.07551 [cs.CR]
- [12] A. Sanna. 2026. AgentDefense-Bench. <https://github.com/arunsanna/AgentDefense-Bench>.
- [13] Snyk. 2026. agent-scan (formerly Invariant Labs MCP-Scan). Vendor tooling.
- [14] Z. Wang, Q. Chang, H. Patel, S. Biju, C.-E. Wu, Q. Liu, A. Ding, A. Reza-zadeh, A. Shah, Y. Bao, and E. Siow. 2025. MCP-Bench: Benchmarking Tool-Using LLM Agents with Complex Real-World Tasks via MCP Servers. arXiv:2508.20453 [cs.AI]
- [15] Z. Wang, Y. Gao, Y. Wang, S. Liu, H. Sun, H. Cheng, G. Shi, H. Du, and X. Li. 2025. MCPTox: A Benchmark for Tool Poisoning Attack on Real-World MCP Servers. arXiv:2508.14925 [cs.CR]
- [16] D. Zhang, Z. Li, X. Luo, X. Liu, P. Li, and W. Xu. 2025. MCP Security Bench (MSB): Benchmarking Attacks Against Model Context Protocol in LLM Agents. arXiv:2510.15994 [cs.CR] ICLR 2026.